

Constraint Type	Constraint Dimension
Content Constraint	Theme Constraint Exclusion Constraint Inclusion Constraint Value Constraint Privacy Constraint Numerical Constraint
Situation Constraint	Role-Playing Constraint Target Audience Constraint Prior Condition Constraint Natural Language Process Background Information Constraint Markdown Process Background Information Constraint Table Background Information Constraint Text Background Information Constraint
Style Constraint	Tone and Style Constraint Emotion Constraint Linguistic Characteristics Constraint Multilingual Constraint
Format Constraint	Output Format Constraint Text Pattern Constraint Grammar Structure Constraint Citation Constraint Numbering and List Constraint Hierarchical Structure Constraint Template Constraint

Table 8: Constraints and Their Dimensions

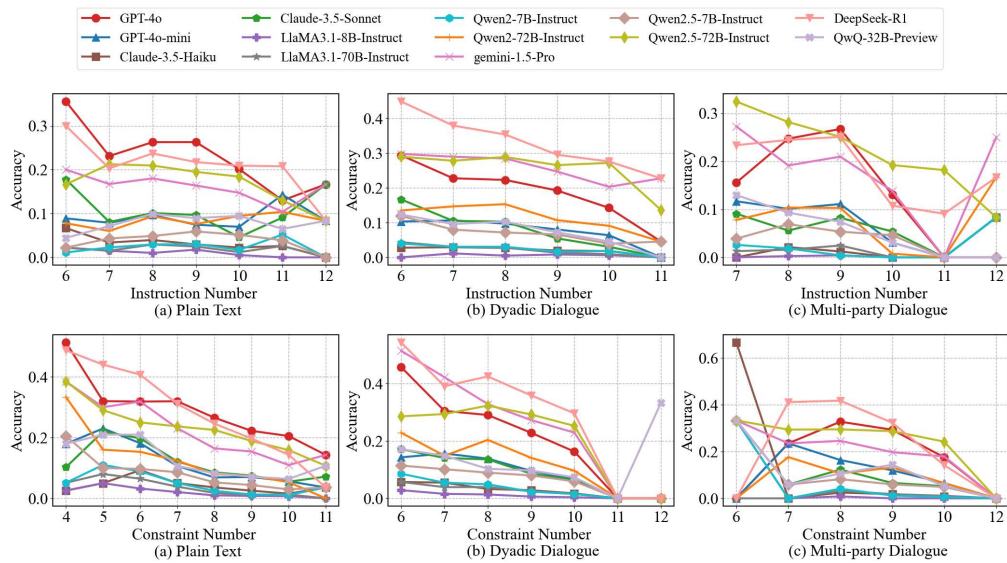


Figure 6: Performance on different numbers of instructions and constraints.

E Prompt

E.1 Prompt for task expansion

You are an assistant to help generate comprehensive multi-task tasks from basic tasks/basic texts/basic dialogues. Based on the given basic task, please design 5-10 different types of extended tasks, which must be reasonable and meet actual needs. The generated tasks should be placed after “-output-”.

Please follow these rules when generating tasks:

1. Task design must be based on the input text content or the already designed task output content.

2. Task instructions should be clear and specific.

3. Each task should include explicit output format requirements.

5. Aim to increase task difficulty, selecting tasks that require multi-step reasoning and thinking.

6. Tasks should be related; specific task details can vary. The connections can be selective, sequential, parallel, etc. At least three types of connections are needed, including:

A. Parallel Task Mode: Analyzing multiple dimensions simultaneously

B. Sequential Task Mode: Task chain dependency

C. Conditional Selection Mode: Branch based on different situations, considering possible branches of the task, and design different tasks for different branches

D. Nested Task Mode: Hierarchical task structure

7. Task Design Principles:

- Clear goals
- Clear instructions
- Specific steps
- Standardized format
- Evaluability

8. Task types may repeat, but task content may not.

9. Note that expansion must be based on the given basic task, expanding into richer, more comprehensive, and varied integrated tasks. The text material in the given basic task must be retained, as subsequent tasks will all involve it!

10. Write the extended tasks after “-output-”, and the thought process and analysis for generating the extended tasks after “-explanation-”.

-input-:
{text}

-output-:

-explanation-:
...

Task-type examples should be chosen from the following categories. The specific task examples are listed in each task category. Note that you are only required to design tasks, not provide example outputs:

1. Classification Task

- Sentiment Analysis
- Text Classification
- Toxic Content Detection
- Empathy Detection
- Stereotype Detection
- Social Norm Judgment

2. Information Extraction

- Named Entity Recognition
- Keyphrase Annotation
- Coreference Resolution
- Entity Relationship Classification

3. Text Generation - Story Creation

- Poetry Generation
- Recipe Generation
- Outline Generation
- Text Expansion/Compression
- Title Generation
- Data Description Generation
- Text Rewriting/Simplification

4. Dialogue Systems

- Dialogue Generation
- Intent Recognition
- Question Generation/Rewriting
- Dialogue State Tracking
- Role-playing Dialogue

5. Reasoning and Logic

- Common Sense QA
- Multi-hop QA
- Critical Thinking Judgment
- Mathematical Reasoning
- Theory of Mind Reasoning

6. Language Style

- Style Transfer
- Language Detection
- Sarcasm Detection
- Spelling/Punctuation Error Detection

7. Evaluation and Verification

- Text Quality Evaluation
- Fact-checking
- Answer Verification
- Uncertainty Judgment

8. Programming-related

- Code Generation/Debugging

- Code Explanation
- Code Translation

Each example format is as follows:

Task x:
Type:
Specific Requirements:
- ...
- ...

E.2 Prompt for task revision

You are a task optimization expert. Please analyze and optimize the given task set.

Input text and tasks are as follows:

```
--input--:
query: {input_text}
task: {task}
```

First, output all optimized tasks (if there are no modifications, output the original tasks) in Chinese after “-output-”. Secondly, write the optimized rationale and analysis process after “-explanation-”. Please strictly follow this format.

The output format is as follows:

```
-output-:
Task 1: ...
Task 2: ...
```

```
-explanation-:
```

Please follow these steps for analysis and optimization:

1. Input Analysis

Input Type Judgment:

- Determine if it's a complete text or a task requirement
- Check if it includes a creative/analytical directive
- Assess the amount of information provided by the text/task
- Preserve textual information if input text analysis is involved

2. Task Reasonability Check Analyze each task:

Reasonability of Task X:

A. Matching Degree with Input

- Does the task rely on the actual provided information, and is there excessive speculation or extension?

B. Executability of the Task

- Is there sufficient information to support it, and are the scoring criteria operable?

C. Existing Problems

- [List specific problems]

3. Modification Suggestions

If modification is required, provide specific modification directions and design of revised tasks, and modify the tasks according to this suggestion.

E.3 Prompt for task combination

You are an integration assistant for input and task requirements. Your goal is to combine the basic tasks (including tasks and reading materials) given in “-input-”: and the expanded tasks in “-task-”: to generate comprehensive tasks that include reading material information and task information. Please note that the integrated tasks will not fetch text information from elsewhere, so ensure that the generated comprehensive tasks include the text material. Please integrate all tasks from the expanded tasks into the comprehensive tasks. Identify all expanded tasks and ensure the number of sub_instructions matches the number of sub-tasks in the expanded tasks. Please ensure to extract and integrate the materials and text information involved in -input-:, and do not omit any details.

```
--input--:
{input_text}
--tasks--:
{task}
```

Please put the generated content in Chinese after “-output-:”, including:
1. First, place the comprehensive task and the text materials involved in the task after “INSTRUCTION:”. Please ensure to fully include the reading materials from the basic tasks.
2. Then, sequentially output all sub-instructions, each starting with “SUB_INSTRUCTION_X:”, including “instruction:” and “constraints:” parts. After “instruction:”, write the content of the sub-instruction, and after “constraints:”, write several constraint items. Each constraint should follow the format “- constraint content [constraint type]”. Lastly, provide the specific combination process after “-explanation-:”.

The output format is as follows:

```
-output-:
INSTRUCTION:
...
SUB_INSTRUCTION_x:
instruction: ...
constraints:
- ... [...]
- ... [...]
```

```
--explanation--:
...
```

Please follow these steps for analysis and combination:

1. Input Analysis

- Extract ****original tasks**** and corresponding ****text materials**** from -input-
- Extract specific text content and basic requirements
- Extract all specific requirements from the input text
- If the input involves text, it must be placed in the comprehensive task to avoid missing the input text

2. Task Expansion Analysis

- Extract ****sub-task**** information (information type, information volume, target) from -task-
- Retrieve related expanded tasks (such as information extraction, reasoning, etc.)
- Understand the relevance and progression relationship between tasks
- Identify all constraints and restrictions
- Record keywords and special conditions
- Note that all tasks are prefixed with "Task", be sure to identify all tasks, and the number of tasks should match the number of sub_instructions
- List all tasks and their sub-tasks

3. Combine into a New Comprehensive Task

- Expand ****original tasks****, ****text materials****, and all ****sub-tasks**** into a comprehensive analysis task, and output to INSTRUCTION
- Maintain logical connections between tasks
- Ensure the INSTRUCTION meets all sub-task requirements
- Make sure to integrate all tasks and incorporate the input
- Ensure all original requirements are covered

4. Integrated Output

- A unified main instruction, output the new comprehensive task (INSTRUCTION):
 - * Use natural language connectors, the task requirements should be connected with natural language, such as "then", "next", "finally", to maintain fluency
 - * Ensure the integration of input and task, the combined content should be output to INSTRUCTION, forming a coherent and smooth instructional language
 - * Ensure all requirements are covered
- A series of sub-instructions (SUB_INSTRUCTION)
 - * Each sub-instruction contains specific tasks (instruction)
 - * Each sub-instruction includes specific constraints (constraints), generating 5-10 specific constraints
 - * The purpose of the constraints is to complete the task as much as possible, the more detailed, the better, with difficulty ranging from simple to

complex

* Note that constraints should not be ambiguous or unclear

* Each constraint and its type should be selected from the following 24 types:

- Theme Constraint
- Exclusion Constraint
- Inclusion Constraint
- Value Constraint
- Privacy Constraint
- Numerical Constraint
- Role-Playing Constraint
- Target Audience Constraint
- Prior Condition Constraint
- Natural Language Process Background
- Markdown Process Background
- Table Background Information
- Text Background Information
- Tone and Style Constraint
- Emotion Constraint
- Linguistic Characteristics
- Multilingual Constraint
- Output Format Constraint
- Text Pattern Constraint
- Grammar Structure Constraint
- Citation Constraint
- Numbering and List Constraint
- Hierarchical Structure Constraint
- Template Constraint

Precautions:

1. Sub-tasks must be clearly mentioned in the integrated instruction.
2. Do not change the wording and expressions of the original instructions.
3. Split according to the order in which the tasks appear in the instructions.
4. Each sub-task is equipped with 5-10 constraint items, with constraint types selected from the above 24 types.
5. When integrating, ensure that new tasks are organically combined with the original content, i.e., do not generate instructions based only on information from input.

E.4 Prompt for constraint expansion

You are an expert at generating constraints.

Please modify the original constraint information for each instruction.

For every SUB_INSTRUCTION, generate 6-10 high-quality constraints.

Each constraint must address key requirements of the task with measurable analysis rather than general statements.

Input information is as follows:

```
--input--:
{input_text}
```

When outputting content, please place the generated content after “-output-” first.

Begin with “INSTRUCTION”, followed by each sub-instruction sequentially, with each sub-instruction starting with “SUB_INSTRUCTION_X”, including both “instruction” and “constraints”.

After “instruction”, write the content of the sub-instruction, and after “constraints”, write several constraint items.

Each constraint should follow the format “- constraint content [constraint type]”.

Finally, place the analysis of the modification process after -explanation-.

The generated format is as follows:

```
-output-:
INSTRUCTION:
...
```

```
SUB_INSTRUCTION_0:
instruction: ...
constraints:
- ... [...]
- ... [...]
...
```

```
SUB_INSTRUCTION_1:
instruction: ...
constraints:
- ... [...]
...
```

```
--explanation--:
...
```

Specific modification requirements:

1. Each constraint must be specific and clear, avoiding vague expressions, and the constraint structure should use “and,” “or,” “not” types.

2. Each constraint must include measurable standards, such as specific numbers, clear criteria, etc.

Also, note that these constraints are for the model to follow, avoiding situations that are impossible to assess, such as “Please respond within 5 seconds after reading,” which cannot be evaluated for compliance.

3. Avoid generic vocabulary; examples below:

Avoid using generic words often found in constraints:

Quality Descriptors:

“appropriate,” “suitable,” “adequate,” “sufficient,” “complete,” “detailed,” “accurate,” “clear,” “varied”

Logical Descriptors:

“logicality,” “coherent,” “orderly,”

“hierarchical,” “structured,” “systematic”

Effect Descriptors:

“comprehensive,” “practical,” “vivid,” “specific,” “pictorial,” “persuasive,” “effective,” “helpful”

Standard Descriptors:

“meets requirements,” “standard-compliant,” “sufficient,” “as stipulated,” “qualified,” “standard fit”

Feature Descriptors:

“characteristic,” “feature,” “prominent,” “obvious,” “outstanding”

These words should be replaced with specifically measurable standards, for example:

“suitable” -> “must include 3 specific examples”

“detailed” -> “no less than 100 words”

“vivid” -> “must use more than 3 figures of speech”

“logicality” -> “must follow [cause-process-result] order”

“persuasive” -> “must cite 1 authoritative data source”

4. Each constraint must be of one of the following types:

- Theme Constraint
- Exclusion Constraint
- Inclusion Constraint
- Value Constraint
- Privacy Constraint
- Numerical Constraint
- Role-Playing Constraint
- Target Audience Constraint
- Prior Condition Constraint
- Natural Language Process Background
- Markdown Process Background
- Table Background Information
- Text Background Information
- Tone and Style Constraint
- Emotion Constraint
- Linguistic Characteristics
- Multilingual Constraint
- Output Format Constraint
- Text Pattern Constraint
- Grammar Structure Constraint
- Citation Constraint
- Numbering and List Constraint
- Hierarchical Structure Constraint
- Template Constraint

E.5 Prompt for constraint revision

You are an assistant for modifying constraints.

Please analyze the original constraint information in the instruction for potential issues, and modify the constraints for each SUB_INSTRUCTION to generate 6-10 high-quality constraints.

Each constraint must address specific, measurable requirements for the task, rather than general statements.

--input--:
{input_text}

When outputting content, first combine the modification analysis process and output the modified content (if no modifications, output the original content) after "-output-", including INSTRUCTION and the modified SUB_INSTRUCTION information, where each SUB_INSTRUCTION consists of instruction and constraints. Each constraint should follow the format "- specific constraint content [constraint type]". Finally, provide the analysis of the modification process after -explanation-.

The generated format is as follows:

-output-:
INSTRUCTION:
...

SUB_INSTRUCTION_x:
instruction: ...
constraints:
- ... [...]
- ... [...]
...

-explanation-:
...

Please follow these steps for analysis and modification:

1. Examine each instruction for the following 8 types of issues and modify any issues found

1.1 Vague constraints/lack of specific evaluation metrics need to be detailed into evaluable metrics

Example:

Original constraint:

- The article structure must be reasonable

Modified to:

- The article must include introduction, analysis, and conclusion sections, with each section not less than 200 words

Below are frequently used vague words that should be avoided:

Quality Descriptors: "appropriate", "suitable", "adequate", "sufficient", "complete", "detailed", "accurate", "clear", "varied"

Logical Descriptors: "logicality", "coherent", "orderly", "hierarchical", "structured", "systematic"

Effect Descriptors: "comprehensive", "practical", "vivid", "specific",

"pictorial", "persuasive", "effective", "helpful"

Standard Descriptors: "meets requirements", "standard-compliant", "sufficient", "as stipulated", "qualified", "standard fit"

Feature Descriptors: "characteristic", "feature", "prominent", "obvious", "outstanding"

First, check if any vague words appear in the constraints, then refine the vague constraints into evaluable metrics based on the specific task context. Here are some examples:

"suitable" -> "must include 3 specific examples"

"detailed" -> "no less than 100 words"

"vivid" -> "must use more than 3 figures of speech"

"logicality" -> "must follow [cause-process-result] order"

"persuasive" -> "must cite 1 authoritative data source"

"rich emotional color" -> Use at least two rhetorical devices (parallelism, contrast, metaphor, personification, or exaggeration) to express emotions

1.2 Duplicate constraints need to be distinguished

Example:

Original constraint:

- Must use formal language

- Must use standard language

Problem: The two constraints are similar and lack distinction

Modification suggestion:

- Must use honorific words like "you, your, respectfully"

- Must avoid using interjectory words like "oh, ah, um"

1.3 Logical contradictions

Example:

Original constraint:

- Both "relaxed and gentle" and "professional terminology" require a remedy

Suggested modification:

- The tone must be friendly and professional, with easy-to-understand explanations provided for professional terminology

1.4 Lack of key constraints

Example:

E-commerce customer service scenario

Suggested modification:

- Must explain the shop's specific compensation plan

- Must provide direct contact details for customer service

- Must specify the follow-up timeline

Original constraint:

- Modify according to the following format

Modification suggestion:

- Modify according to the table format

1.5 Contradictory constraints:

Original constraint:

- Requires classical Chinese style
 - Requires vividness
- Suggestion: Adjust to:
- Use classical vocabulary but ensure modern readers can understand
 - Provide modern explanations for each term

1.6 Lack of key definitions:

Original constraint:

- The calculation of the number of “events” lacks a clear definition
- Suggestion: Add:
- Clearly define “event” as “an independent action and its corresponding object”
 - Provide specific examples for event judgment

1.7 Data source missing

Original constraint:

- “Must include specific data or factual references”
 - “Must be based on specific data and facts”
- But no instructions on how to obtain and verify data sources
- Suggestion: Add data source requirements:
- “Must cite authoritative market research agencies or official publications, and specify the source”
 - “Data must be from statistical results within the past 2 years”

1.8 Evaluation criteria unclear:

Original constraint:

- “Applicable scenarios must be reasonable and consistent with market reality”
 - “Usage suggestions must be specific and feasible”
- But no evaluation criteria provided
- Suggestion: Set specific evaluation indicators:
- “Each suggestion must include usage scenarios, expected effects, and cost considerations”
 - Add feasibility verification:
 - “Each suggestion must be supported by actual cases”

2. Constraint types should be selected from the following 24 categories while varying the types as much as possible to enrich the diversity of the constraints:

- Theme Constraint
- Exclusion Constraint
- Inclusion Constraint
- Value Constraint
- Privacy Constraint
- Numerical Constraint

- Role-Playing Constraint
- Target Audience Constraint
- Prior Condition Constraint
- Natural Language Process Background
- Markdown Process Background
- Table Background Information
- Text Background Information
- Tone and Style Constraint
- Emotion Constraint
- Linguistic Characteristics
- Multilingual Constraint
- Output Format Constraint
- Text Pattern Constraint
- Grammar Structure Constraint
- Citation Constraint
- Numbering and List Constraint
- Hierarchical Structure Constraint
- Template Constraint

E.6 Prompt for constraint combination

Now you are an assistant in integrating tasks and constraints; please help me optimize this comprehensive task’s instruction (INSTRUCTION), sub-instructions (SUB_INSTRUCTION), and constraints.

Please ensure that the input information/reading materials in the INSTRUCTION are retained; otherwise, subsequent tasks cannot be completed.

Input information is as follows:

```
--input--:
{input_text}
```

First, output the modified comprehensive task content (if no modifications, output the original content) after “-output-”, including INSTRUCTION and the modified SUB_INSTRUCTION information. Each SUB_INSTRUCTION consists of instruction and constraints, with each constraint structured as follows: “- specific content [constraint type]”.

Finally, put the specific modification analysis process after -explanation-.

The output format is as follows:

```
-output-:
INSTRUCTION:
...

SUB_INSTRUCTION_0:
instruction: ...
constraints:
- ... [...]
- ... [...]
...

SUB_INSTRUCTION_1:
instruction: ...
constraints:
```

- ... [...]
- ... [...]
...

-explanation:-
...

Please follow these steps for analysis and modification:

1. Analyze the existing instructions and constraints for issues:
 - Check if the structure is reasonable
 - Identify duplicate or contradictory requirements
 - Discover vague or non-executable constraints
 - Find missing key requirements
2. Provide update suggestions:
 - Instruction update: Make it clearer and more targeted for the comprehensive task
 - Sub-instruction update: Specify each atomic task
 - Constraint update: Provide executable and verifiable constraints
 - Start with -output-, output the modified instructions (INSTRUCTION), sub-instructions (SUB_INSTRUCTION), and constraints
 - Each constraint includes two parts: content [type]
3. When modifying, be sure to keep the input information such as reading materials in the original INSTRUCTION. Do not delete specific query information, causing text errors.
4. Each constraint type must be one of the following, if it is not among these types, please modify the constraint type to one of the following types. If it cannot be modified, regenerate constraints that meet these types:

- Theme Constraint
- Exclusion Constraint
- Inclusion Constraint
- Value Constraint
- Privacy Constraint
- Numerical Constraint
- Role-Playing Constraint
- Target Audience Constraint
- Prior Condition Constraint
- Natural Language Process Background
- Markdown Process Background
- Table Background Information
- Text Background Information
- Tone and Style Constraint
- Emotion Constraint
- Linguistic Characteristics
- Multilingual Constraint
- Output Format Constraint
- Text Pattern Constraint
- Grammar Structure Constraint
- Citation Constraint

- Numbering and List Constraint
- Hierarchical Structure Constraint
- Template Constraint

E.7 Prompt for instruction-level validation

You are now an assistant to modify sub-tasks.

You need to modify the given sub-tasks according to the following steps:

1. Analyze the relationship between sub-tasks and evaluate their role in the comprehensive task, removing contradictory sub-tasks.
2. Note that sub-tasks are carried out by a large model, so remove tasks that the large model cannot complete, such as internet searches, finding related information, statistical data analysis, etc.
3. Delete tasks with weak logical connections. The relationships between sub-tasks can be:
 - A. Parallel Task Mode: Analyzing multiple dimensions simultaneously
 - B. Serial Task Mode: Chain-dependent tasks
 - C. Conditional Selection Mode: Branching based on different situations, considering possible branches of a task, and designing different tasks for different branches
 - D. Nested Task Mode: Hierarchical task structure
4. Sub-task selection criteria:
 - Remove tasks that an AI model cannot accomplish (such as network searches, finding information)
 - Remove tasks with weak logical connections
 - Remove redundant, contradictory, or unreasonable tasks
 - Optimize sub-task content to be of moderate difficulty and meet practical needs
 - It is acceptable to propose some creative tasks
 - Ensure that the number of generated sub-tasks is between 6 and 14
 - Try to diversify task types, with at least 3 different styles of tasks
 - Remove tasks that require an AI model to use tools, such as Named Entity tools, etc.
5. Select the main task categories from the following, and the directions under each category can be diversified:
 1. Classification Task
 - Sentiment Analysis
 - Text Classification
 - Toxic Content Detection
 - Empathy Detection
 - Stereotype Detection
 - Social Norm Judgment
 2. Information Extraction
 - Named Entity Recognition

- Keyphrase Annotation
- Coreference Resolution
- Entity Relationship Classification

3. Text Generation - Story Creation

- Poetry Generation
- Recipe Generation
- Outline Generation
- Text Expansion/Compression
- Title Generation
- Data Description Generation
- Text Rewriting/Simplification

4. Dialogue Systems

- Dialogue Generation
- Intent Recognition
- Question Generation/Rewriting
- Dialogue State Tracking
- Role-playing Dialogue

5. Reasoning and Logic

- Common Sense QA
- Multi-hop QA
- Critical Thinking Judgment
- Mathematical Reasoning
- Theory of Mind Reasoning

6. Language Style

- Style Transfer
- Language Detection
- Sarcasm Detection
- Spelling/Punctuation Error Detection

7. Evaluation and Verification

- Text Quality Evaluation
- Fact-checking
- Answer Verification
- Uncertainty Judgment

8. Programming-related

- Code Generation/Debugging
- Code Explanation
- Code Translation

Input Total Task:
{input_text}

Input Sub-tasks:
{sub_instruction}

6. First, output the modified comprehensive task content (if no modifications, output the original content) after -output-, including the modified INSTRUCTION and SUB_INSTRUCTION_x, where SUB_INSTRUCTION_x is formatted as 'sub-task content [task type]'. Finally, place the specific modification analysis process after -explanation-.

-output-:
INSTRUCTION:
...

SUB_INSTRUCTION_0:

... [task type]

SUB_INSTRUCTION_1:
... [task type]

-explanation-:
...

E.8 Prompt for constraint-level validation

You are a constraint evaluation assistant.

Your task is to determine whether the given constraints can be completed by a large model. Please evaluate according to the following rules:

1. **Input**:

- Constraint content: A segment of text describing the task requirements.
- Input dialogue: A segment of the user's conversation with the model.

2. **Evaluation Rules**:

- If the input dialogue **lacks** the critical information needed to fulfill the constraint (e.g., the constraint requires extracting person entities, but no person is mentioned in the dialogue), then output "No".
- If the constraint **goes beyond** the model's capability (e.g., needs real-time data or external resources), then output "No".
- If the input dialogue provides sufficient information and the constraint falls within the model's capability, then output "Yes".
- If the model outputs "No", minimally modify the constraint content to make it feasible for the model to complete it.

3. **Output**:

- If the output is "No", provide the modified constraint content to make it feasible for the model.
- If the output is "Yes", no modification is needed.

4. **Examples**:

- Example 1:
 - Instruction content: Extract entities
 - Constraint content: Extract person entities from the dialogue.
 - Input dialogue: User says, "Yesterday I went to the park with Xiaoming."
 - Output: Yes
- Example 2:
 - Instruction content: Extract entities
 - Constraint content: Extract person entities from the dialogue.
 - Input dialogue: User says, "The weather was great yesterday, and I went for a walk in the park."
 - Output: No
 - Reason: No person entities in the dialogue
 - Modified content: If any person

entities are present, extract them.

- Example 3:
- Instruction content: Generate text
- Constraint content: Generate a 100-word text describing the summer scenery, using at least 3 similes.
- Input dialogue: User says, "Please write a passage about summer."
- Output: Yes
- Example 4:
- Instruction content: Generate text
- Constraint content: Generate a 100-word text describing the summer scenery, and cite at least 2 academic papers.
- Input dialogue: User says, "Please write a passage about summer."
- Output: No
- Reason: Unable to cite academic papers
- Modified constraint: Generate a 100-word text describing the summer scenery, using at least 3 similes.

5. **Task**:

- Instruction content: {instruction}
- Constraint content: {constraint}
- Input dialogue: {input}
- Output:
- Reason:
- Modified constraint:

E.9 Prompt for training process

You are now an AI assistant responsible for generating answers to specified tasks. You need to generate answers following these requirements:

1. Strictly generate answers based on the given input material and corresponding sub_instruction.
2. Generate answers for each sub_instruction, ensuring consistency among answers to different sub_instructions.
3. Follow the constraints of each sub_instruction strictly to generate answers.
4. First, think through each sub-task in detail using analytical skills to deeply understand the issues, and then provide answers. The thought process for each sub-task should be detailed between start_think and end_think, and the answer should be fully presented between start_answer and end_answer.
5. The thought process and answer for each sub_instruction should be placed between start_sub_instruction_x and end_sub_instruction_x, where sub_instruction_x is the specific identifier for the sub-task. Ensure there are no extra spaces, quotes, or symbols before and after these markers.

Notes: 1. Strictly adhere to the constraints.
2. Ensure the quality of answers.

3. Do not output the input content.
4. The format is as follows:

```
start_sub_instruction_0
start_think
Deeply analyze this sub-task, ...
end_think
start_answer
Based on the above analysis, the detailed
answer to sub-task 0 is ...
end_answer
end_sub_instruction_0

start_sub_instruction_1
start_think
In this sub-task, consider various
factors, ...
end_think
start_answer
Based on the above analysis, the answer
to sub-task 1 is ...
end_answer
end_sub_instruction_1

...
```

Referring to the above format and generation requirements, please think through and generate specific answers for the following task:

```
--input--:
{input_text}
--output--:
```